ТЕОРИЯ ГРАФОВ

Изоморфизм графов

Гомоморфизм

Гомоморфизм определяется для алгебр одного типа. Пусть $\mathcal{A} = \langle A; \varphi_1, \ldots, \varphi_m \rangle$ и $\mathcal{B} = \langle B; \psi_1, \ldots, \psi_m \rangle$ — две алгебры одного типа. Если существует функция $f: A \to B$, такая, что

$$\forall i \in 1..m \ \left(f\left(\varphi_i(a_1, \ldots, a_n)\right) = \psi_i\left(f(a_1), \ldots, f(a_n)\right)\right),$$

то говорят, что f — *гомоморфизм* из $\mathcal{A}$ в $\mathcal{B}$.

Пусть $\mathcal{A} = \langle A; \varphi \rangle$, $\mathcal{B} = \langle B; \psi \rangle$, тип $= (1)$ и $f: A \to B$. Действие функций φ, ψ, f можно изобразить с помощью следующей диаграммы:

$$\begin{tikzcd} A \arrow[r, "\varphi"] \arrow[d, "f"'] & A \arrow[d, "f"] \\ B \arrow[r, "\psi"] & B \end{tikzcd}$$

$$f \circ \varphi = \psi \circ f.$$

- Гомоморфизм, который является биекцией, - **изоморфизм**.
- Если A = B , то изоморфизм называется автоморфизмом.

Изоморфизм графов

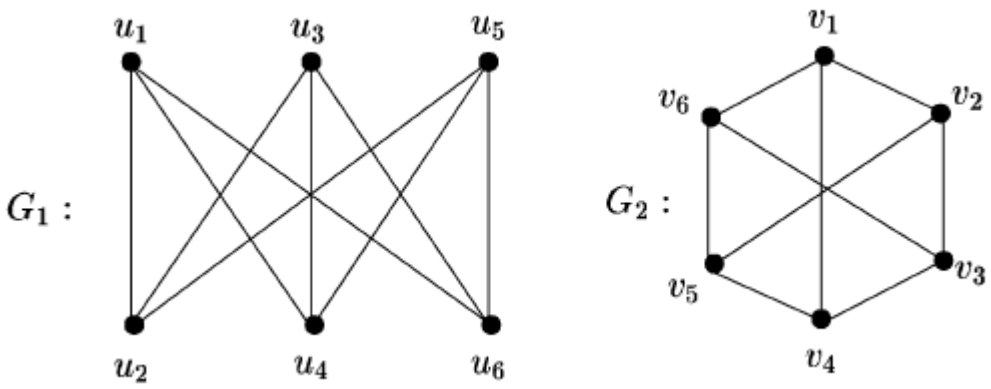
- Говорят, что два графа, *G1(V1,E2)* и *G2(V2,E2)*, *изоморфны* (обозначается *G1 ~ G2)*, если существует биекция *h: V1 —> V2*, сохраняющая смежность:

$$e_1 = (u, v) \in E_1 \Longleftrightarrow e_2 = \big(h(u), h(v)\big) \in E_2.$$

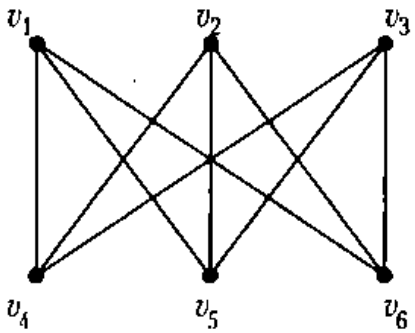
- *Изоморфизм графов есть отношение эквивалентности (*отношение рефлексивно, симметрично и транзитивно, то есть является отношением эквивалентности.*).*
- Графы рассматриваются с *точностью до изоморфизма,* то есть рассматриваются классы эквивалентности по отношению изоморфизма.

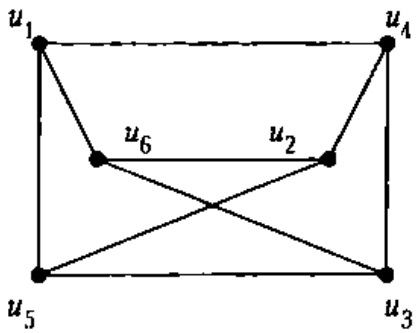
Изоморфизм графов

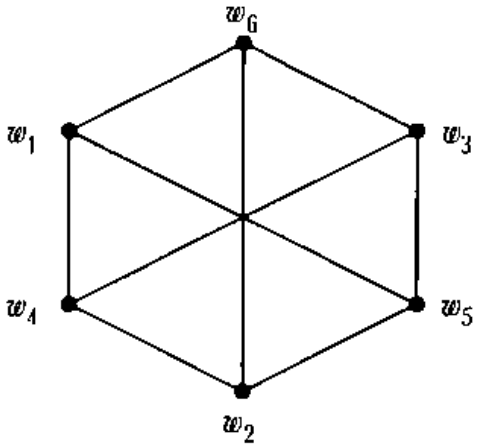
- Отношение изоморфизма на множестве всех графов является отношением эквивалентности.
- Множество всех графов разбивается на классы попарно изоморфных графов.
- Диаграмма задает граф с точностью до изоморфизма.



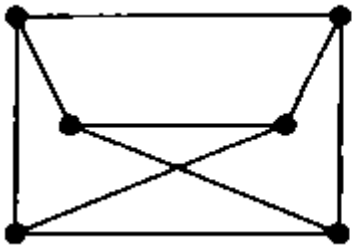
Изоморфизм графов

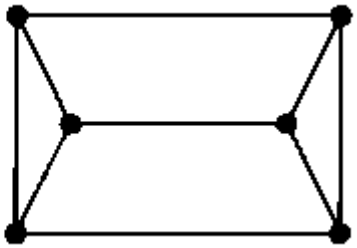






- Числовая характеристика, одинаковая для всех изоморфных графов, называется *инвариантом* графа. Так, *p(G)* и *q(G)* — инварианты графа *G.*
- *Неизвестно никакого простого набора инвариантов, определяющих граф с точностью до изоморфизма.*





Автоморфизм

- Характеристикой симметрии графа является его группа автоморфизмов.
- Произвольная подстановка ϕ на множестве вершин графа *G*, *сохраняющая отношение смежности*, т. е. такая, что образы ϕ(*u*) и ϕ(*v*) вершин *u* и *v* смежны тогда и только тогда, когда смежны сами вершины *u* и *v*, называется *автоморфизмом* графа *G.*
- Иными словами, автоморфизм графа — это изоморфизм графа на себя.
- Любой граф *G* имеет по меньшей мере один автоморфизм — тождественное преобразование *e: VG -> VG*, при котором *e*(*v*)=*v* для любой вершины *v.* Очевидно, что если ϕ — автоморфизм графа *G*, то и обратная подстановка ϕ^{-1} также является автоморфизмом, если же подстановки ϕ и ψ обе суть автоморфизмы, то и их произведение ϕψ — автоморфизм.

2 теоремы об изоморфизме

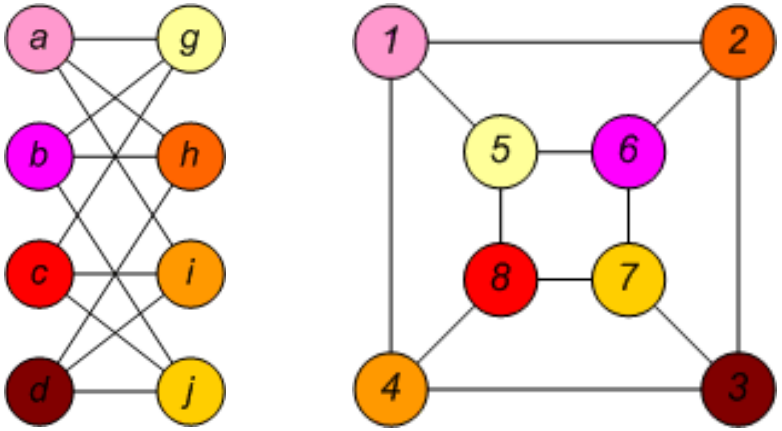
- *Теорема 1.* **Графы изоморфны тогда и только тогда, когда их матрицы смежности вершин получаются друг из друга одновременными перестановками строк и столбцов (т.е. одновременно с перестановкой *i* -й и *j* -й строк переставляются *i* -й и *j* –й столбцы).**

- *Теорема 2.* **Графы (орграфы) изоморфны тогда и только тогда, когда их матрицы инциденций получаются друг из друга произвольными перестановками строк и столбцов.**

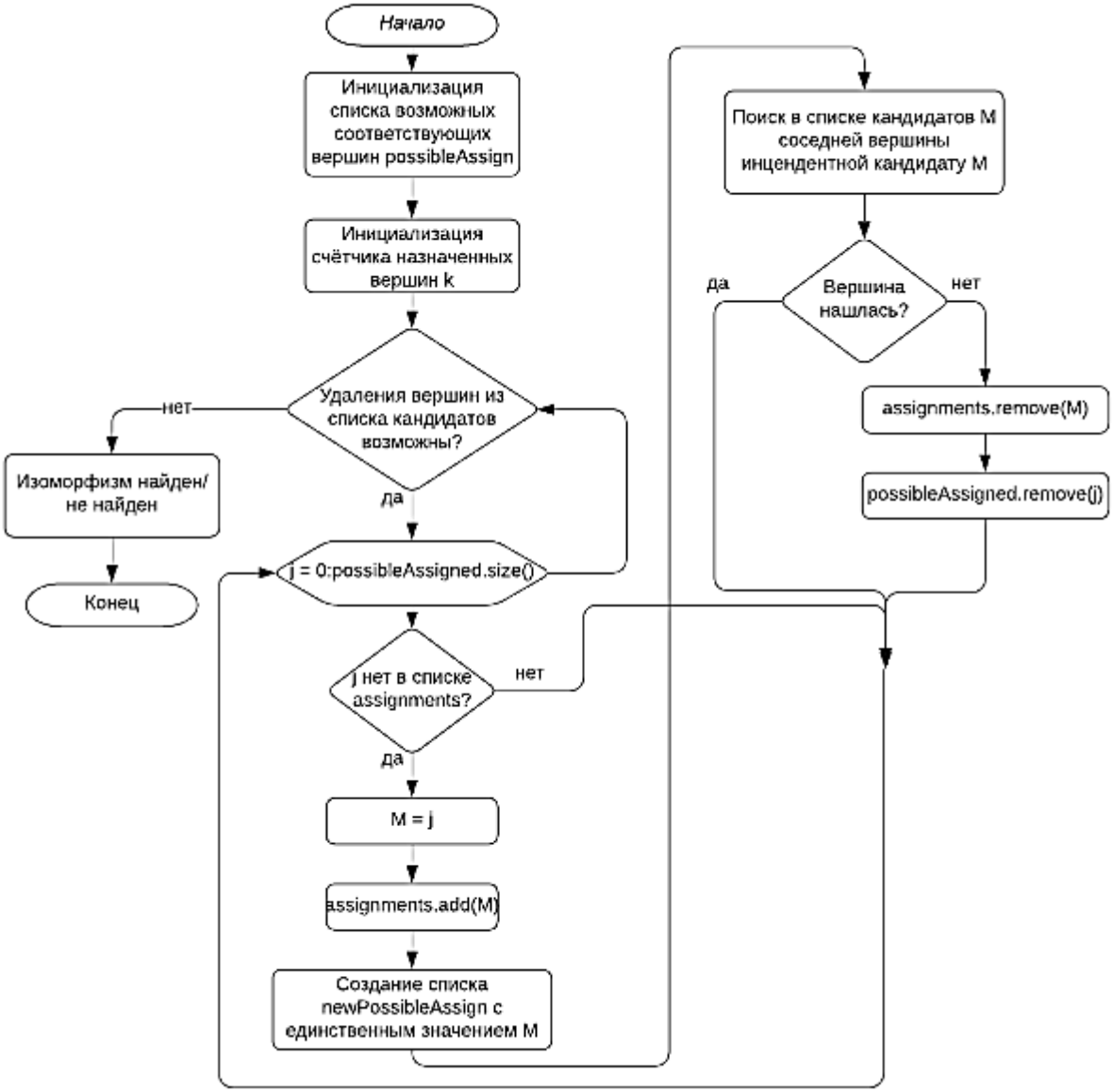
Зачем?

- Поиск в базе данных химических соединений, подобных некоторой заданной молекуле по ее структурной формуле (хемоинформатика); задачи сопоставления с образцом; различные биоинформатические задачи…
- Более сложная задача: поиск в некотором графе всех подграфов, изоморфных некоторому другому графу!

Алгоритм проверки изоморфизма графов



- Классический подход к решению данной задачи был заложен Дж. Ульманом в 1976 году (впоследствии он существенно доработал свой алгоритм).
- Суть этого подхода заключается в «умном переборе» возможных соответствий вершин графа-образца B и графа A, в котором идет его поиск (поиск в глубину). «Умным» этот перебор делает ввод различных условий и ограничений, которые позволяют как можно раньше отсечь неподходящие варианты.
- Также для этих целей может проводиться различная предварительная обработка исходных данных: изначальное упорядочивание вершин графа в порядке убывания степени.
- Решение по данному алгоритму находится за экспоненциальное время $O(n!\,n^3)$.


Начало
Инициализация списка возможных соответствующих вершин possibleAssign
Инициализация счётчика назначенных вершин k
Удаления вершин из списка кандидатов возможны?
нет
Изоморфизм найден/ не найден
Конец
да
j = 0:possibleAssigned.size()
j нет в списке assignments?
нет
да
M = j
assignments.add(M)
Создание списка newPossibleAssign с единственным значением M
Поиск в списке кандидатов M соседней вершины инцендентной кандидату M
Вершина нашлась?
да
нет
assignments.remove(M)
possibleAssigned.remove(j)

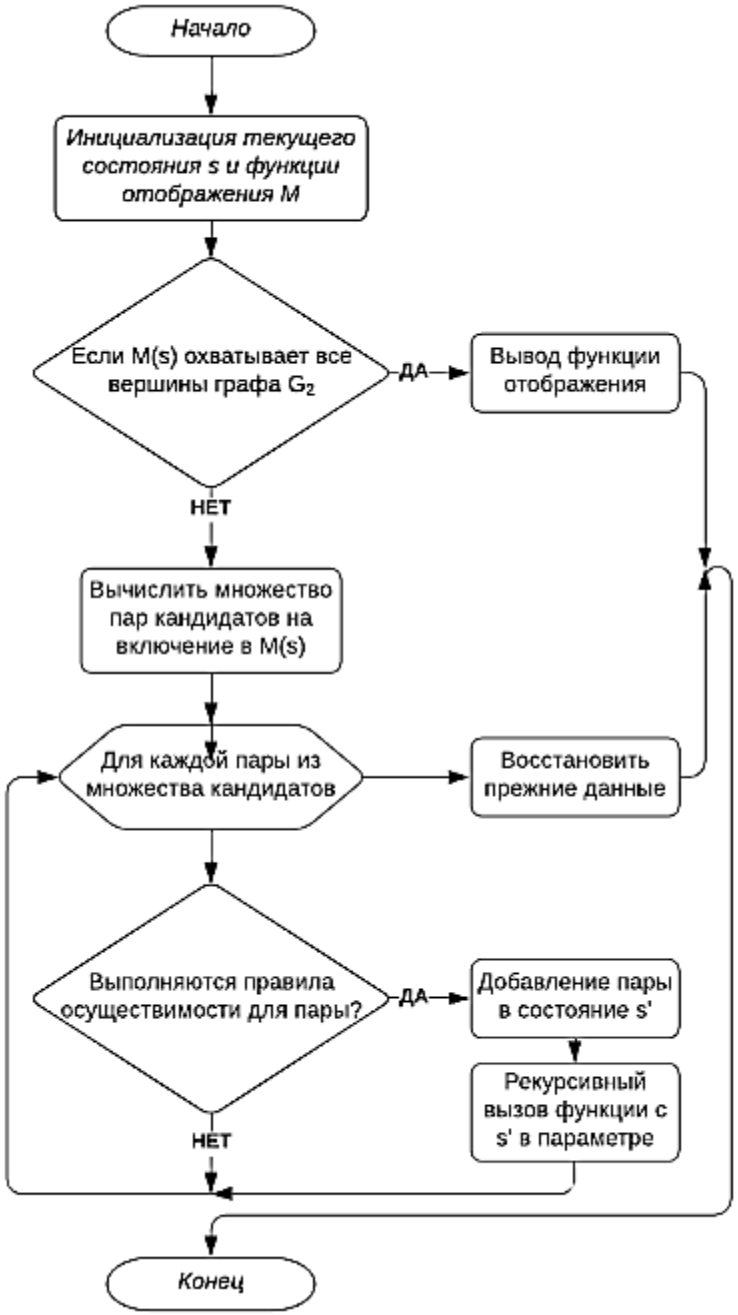
Алгоритм Ульмана

```
def search(graph,subgraph,assignments,possible_assignments):
  update_possible_assignments(graph,subgraph,possible_assignments)

  i=len(assignments)

  # Make sure that every edge between assigned vertices in the subgraph is also an
  # edge in the graph.
  for edge in subgraph.edges:
    if edge.first<i and edge.second<i:
      if not graph.has_edge(assignments[edge.first],assignments[edge.second]):
        return False

  # If all the vertices in the subgraph are assigned, then we are done.
  if i==subgraph.n_vertices:
    return True
```

```
def search(graph,subgraph,assignments):
  i=len(assignments)

  # Make sure that every edge between assigned vertices in the subgraph is also an
  # edge in the graph.
  for edge in subgraph.edges:
    if edge.first<i and edge.second<i:
      if not graph.has_edge(assignments[edge.first],assignments[edge.second]):
        return False

  # If all the vertices in the subgraph are assigned, then we are done.
  if i==subgraph.n_vertices:
    return True

  # Otherwise, go through all the possible assignments for the next vertex of
  # the subgraph and try it.
  for j in possible_assignments[i]:
    if j not in assignments:
      assignments.append(j)
      if search(graph,subgraph,assignments):
        # This worked, so we've found an isomorphism.
        return True
      assignments.pop()

def find_isomorphism(graph,subgraph):
  assignments=[]
  if search(graph,subgraph,assignments):
    return assignments
  return None
```

Алгоритм VF

- Алгоритм VF2 является доработанной версией алгоритма Ульмана, осуществляющего определение изоморфизма графов и подграфов больших размерностей.
- Решение по данному алгоритму находится за экспоненциальное время O(n!n).



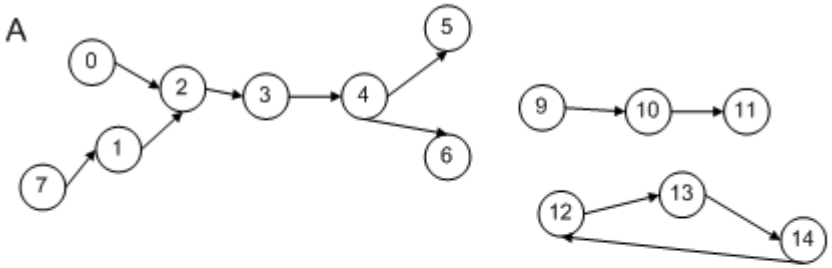
Поиск подграфов

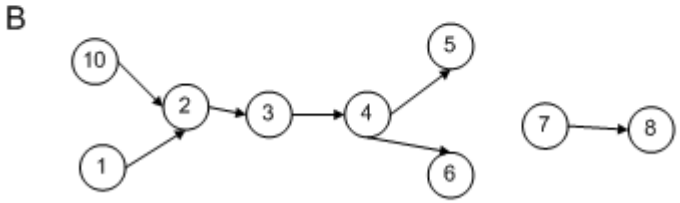
- Можно находить не любые, но лишь ***«вписанные» подграфы***.
- *«Вписанным» подграфом* графа А мы будем называть такой его подграф, который может быть «приклеен» к другим частям графа А только за счет ребер, инцидентных лишь граничным вершинам его (подграфа) неразветвляющихся путей максимальной длины (при этом граф А может содержать и иные компоненты связности).

Пример

Обнаружены следующие соответствия:

- внутренний путь **2->3->4** графа B: внутренний путь **2->3->4** графа A,
- концевые пути **1->2** и **10->2** графа B: концевой путь **0->2** графа A и фрагмент концевого пути **7->1->2** графа A, а именно **1->2**,
- простая цепь **7->8** графа B: отрезки простой цепи **9->10->11** графа A, а именно **9->10** и **10->11**, а также отрезки простого цикла **12->13->14->12** графа A, а именно **12->13, 13->14** и **14->12**.





Пример

- В качестве «изоморфных графу B «вписанных» подграфов графа A могут быть найдены следующие 5 вариантов:

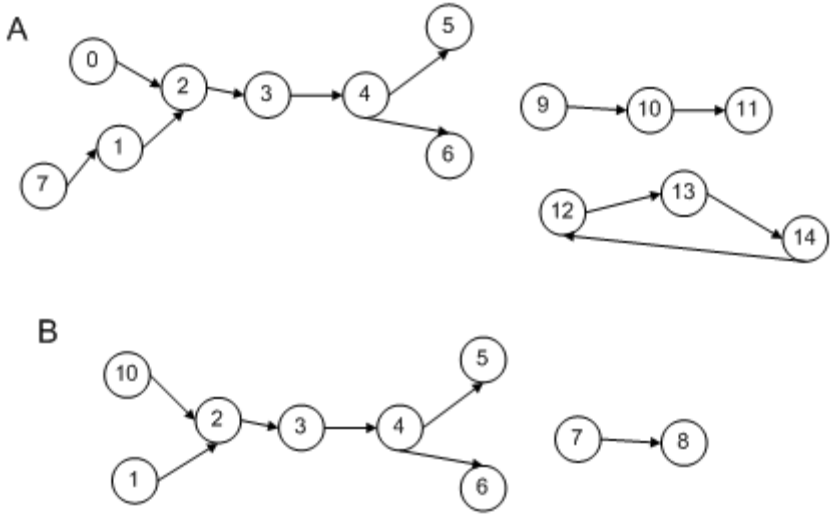
  A1 = {0->2, 1->2, 2->3, 3->4, 4->5, 4->6, 9->10}

  A2 = {0->2, 1->2, 2->3, 3->4, 4->5, 4->6, 10->11}

  A3 = {0->2, 1->2, 2->3, 3->4, 4->5, 4->6, 12->13}

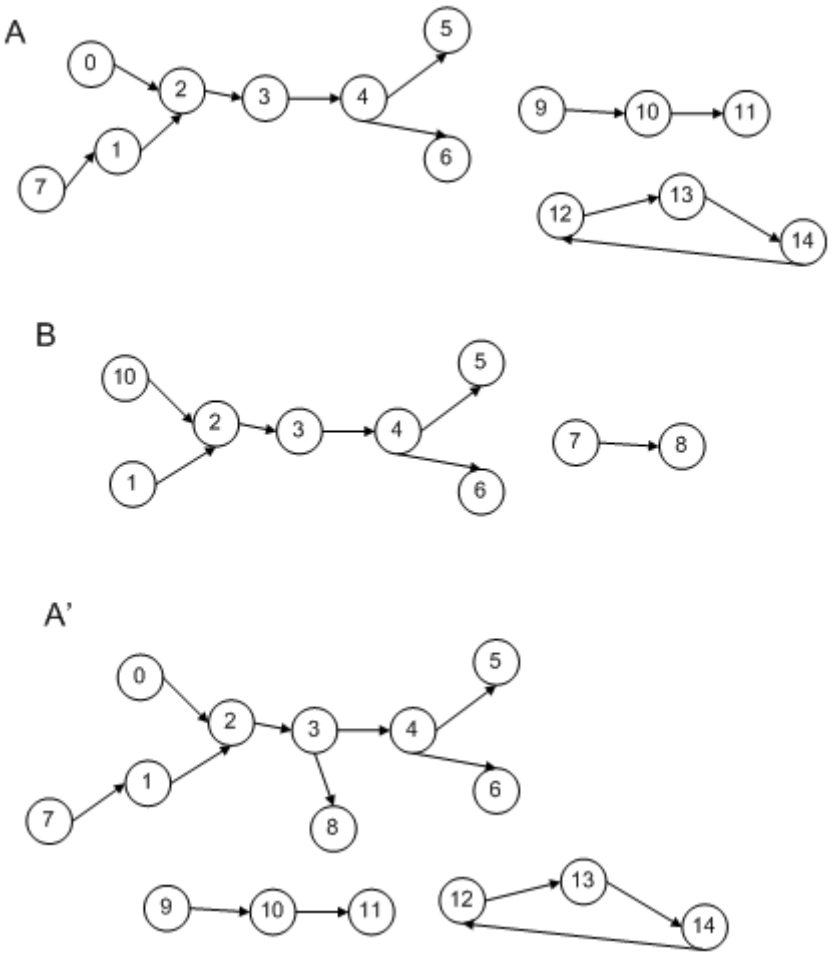
  A4 = {0->2, 1->2, 2->3, 3->4, 4->5, 4->6, 13->14}

  A5 = {0->2, 1->2, 2->3, 3->4, 4->5, 4->6, 14->12}



Пример

- Если добавить к графу А дополнительное ребро **3->8**, получится граф А’.
- В А’ уже будет не найти «вписанных» подграфов, изоморфных графу В.
- Ребро 3->8 «разбивает» путь **2->3->4** графа А на два и, значит, путей-кандидатов для внутреннего пути **2->3->4** графа В обнаружено не будет.



Обходы графов

- Обход графа — это некоторое систематическое перечисление его вершин (и/или рёбер). Наибольший интерес представляют обходы, использующие локальную информацию (списки смежности).
- Среди всех обходов наиболее известны поиск *в ширину* и *в глубину.* Алгоритмы поиска в ширину и в глубину лежат в основе многих алгоритмов на графах.

- **Т Е О Р Е М А.** *Если граф G связен (и конечен), то поиск в ширину и поиск в глубину обходят все вершины по одному разу.*

Поиск в ширину и в глубину

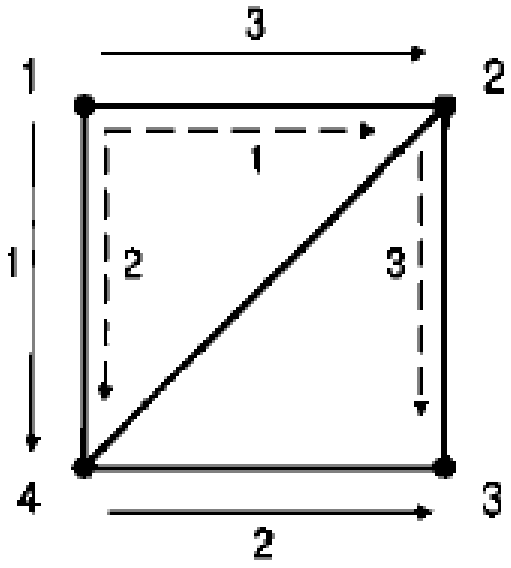
Вход: граф $G(V, E)$, представленный списками смежности Γ.
Выход: последовательность вершин обхода.

```
for v ∈ V do
    x[v] := 0 { вначале все вершины не отмечены }
end for
select v ∈ V { начало обхода — произвольная вершина }
v → T { помещаем v в структуру данных T... }
x[v] := 1 { ...и отмечаем вершину v }
repeat
    u ← T { извлекаем вершину из структуры данных T... }
    yield u { ...и возвращаем её в качестве очередной пройденной вершины }
    for w ∈ Γ(u) do
        if x[w] = 0 then
            w → T { помещаем w в структуру данных T... }
            x[w] := 1 { ...и отмечаем вершину w }
        end if
    end for
until T = ∅
```

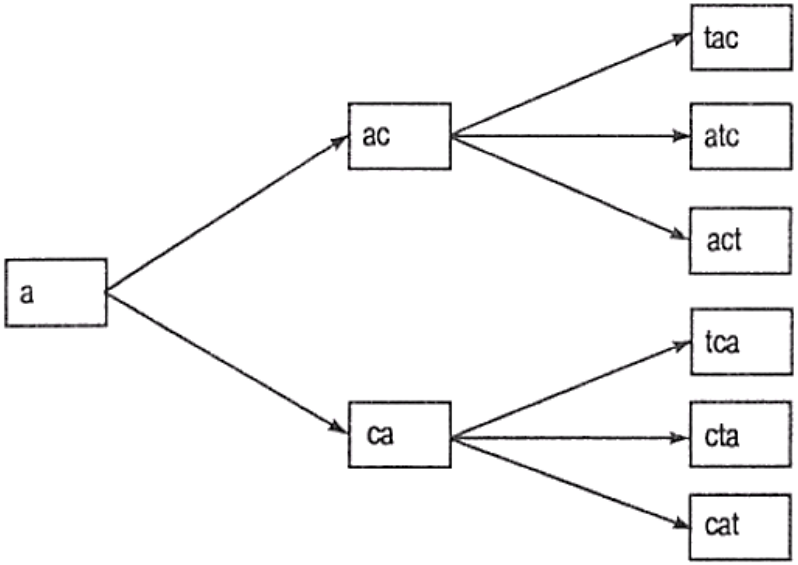
- Если в алгоритме структуры данных T — это стек (LIFO — Last In First Out), то обход называется *поиском в глубину.* Если *T* — это очередь (FIFO — First In First Out), то обход называется *поиском в ширину.*

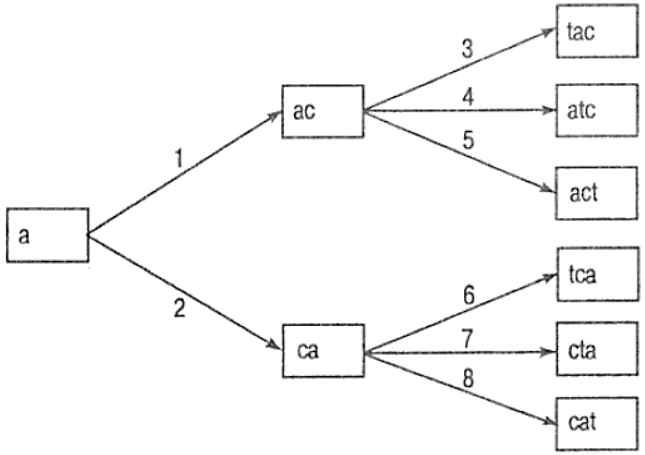
Пример работы алгоритма

u	T	u	T
1	2,4	1	2,4
4	2,3	2	4,3
3	2	4	3
2	∅	3	∅

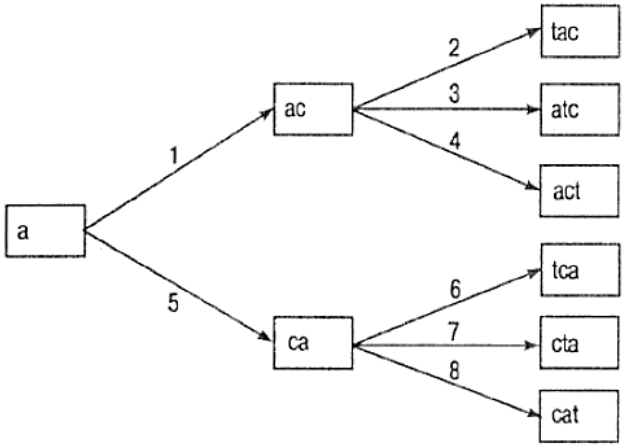


Пример работы алгоритма





поиск в ширину



поиск в глубину

Поиск в ширину (BFS)

```
WhateverFirstSearch(s):
  put s into the bag
  while the bag is not empty
      take v from the bag
      if v is unmarked
          mark v
          for each edge v→w
              put w into the bag
```

```
CountComponents(G):
  count ← 0
  for all vertices v
      unmark v
  for all vertices v
      if v is unmarked
          count ← count + 1
          WhateverFirstSearch(v)
  return count
```

```
CountAndLabel(G):
  count ← 0
  for all vertices v
      unmark v
  for all vertices v
      if v is unmarked
          count ← count + 1
          LabelOne(v, count)
  return count
```

```
⟨⟨Label one component⟩⟩
LabelOne(v, count):
  while the bag is not empty
      take v from the bag
      if v is unmarked
          mark v
          comp(v) ← count
          for each edge vw
              put w into the bag
```

Поиск в глубину (DFS)

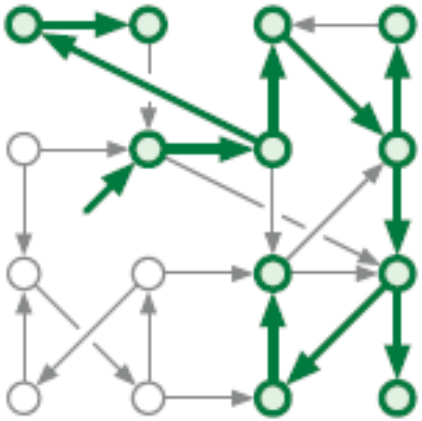
```
WhateverFirstSearch(s):
  put s into the bag
  while the bag is not empty
      take v from the bag
      if v is unmarked
          mark v
          for each edge vw
              put w into the bag
```

```
DFS(v):
  if v is unmarked
      mark v
      for each edge v→w
          DFS(w)
```

```
RecursiveDFS(v):
  if v is unmarked
      mark v
      for each edge vw
          RecursiveDFS(w)
```

```
IterativeDFS(s):
  Push(s)
  while the stack is not empty
      v ← Pop
      if v is unmarked
          mark v
          for each edge vw
              Push(w)
```

Пример обхода в глубину для разных вершин



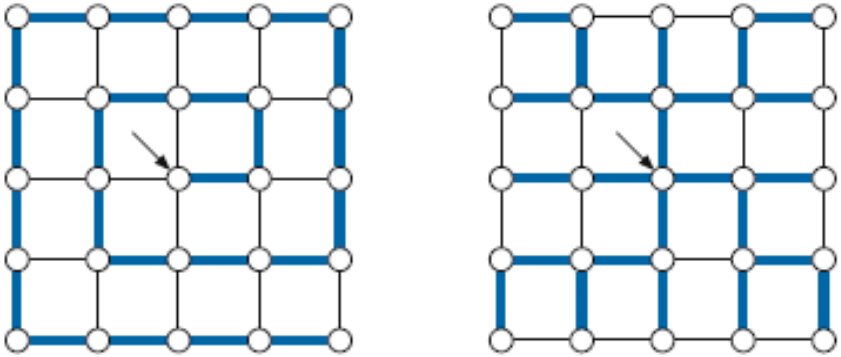
- Обнаружение циклов

```
IsAcyclic(G):
  for all vertices v
      v.status ← New
  for all vertices v
      if v.status = New
          if IsAcyclicDFS(v) = False
              return False
  return True
```

```
IsAcyclicDFS(v):
  v.status ← Active
  for each edge v→w
      if w.status = Active
          return False
      else if w.status = New
          if IsAcyclicDFS(w) = False
              return False
  v.status ← Finished
  return True
```

Поиск в глубину и ширину

- Остовное дерево поиска в глубину и ширину



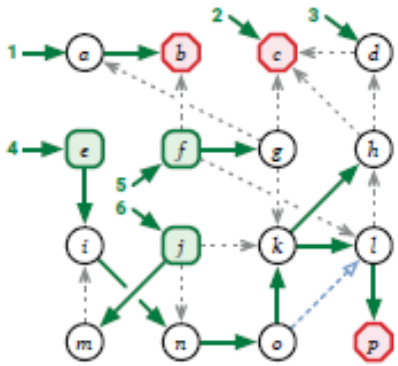
Сборщики мусора (1975 год)

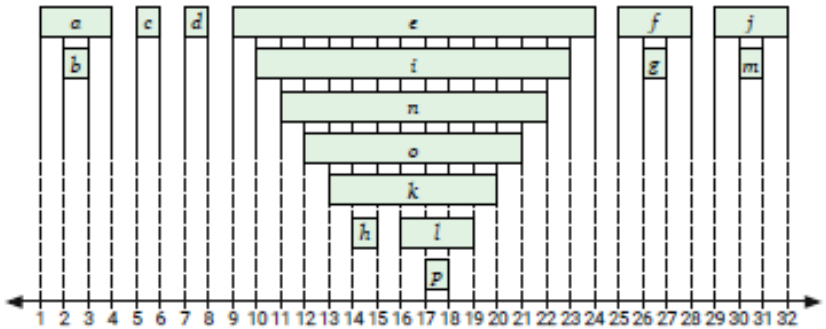
- Вместо отслеживания помеченных и непомеченных вершин алгоритм использовал цвет: белый, серый, черный.
- Все вершины, достижимые из s, черные, не достижимые – белые и родительские ребра являются остовным деревом с корнем компоненты, содержащей s.
- Серые вершины сохраняются в стеке.
- Равнозначен поиску в глубину (но можно реализовать и поиск в ширину).

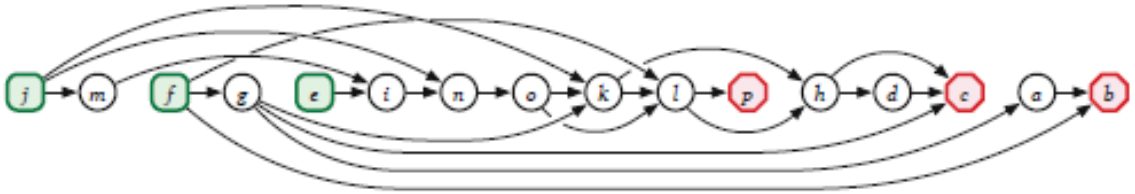
```
THREECOLORSEARCH(s):
    color all nodes white
    color s gray
    while at least one vertex is gray
        THREECOLORSTEP( )
```

```
THREECOLORSTEP( ):
    v ← any gray vertex
    if v has no white neighbors
        color v black
    else
        w ← any white neighbor of v
        parent(w) ← v
        color w gray
```

Топологическая сортировка








```
TopologicalSort(G):
    for all vertices v
        v.status ← New
    clock ← V
    for all vertices v
        if v.status = New
            clock ← TopSortDFS(v, clock)
    return S[1..V]
```

```
TopSortDFS(v, clock):
    v.status ← Active
    for each edge v→w
        if w.status = New
            clock ← TopSortDFS(v, clock)
        else if w.status = Active
            fail gracefully
    v.status ← Finished
    S[clock] ← v
    clock ← clock − 1
    return clock
```

Дерево решений и метод дерева решений

- **Метод дерева решений** применяется в задачах классификации и прогнозирования, когда решения приходится принимать в условиях риска, неопределённости и исход событий зависит от вероятностей. На каждое решение влияют какие-то определённые факторы, и у каждого решения есть свои последствия, которым присущ вероятностный характер. В этих условиях процесс принятия решений является последовательным и **метод дерева решений** предполагает определять, какие действия следует предпринять в каждой вершине дерева.
- **Дерево решений** - математическая модель, которая задаёт процесс принятия решений так, что будут отображены каждое возможное решение, предшествующие и последующие этим решениям события или другие решения и последствия каждого конечного решения.
- Дерево решений состоит из следующих элементов: дуг, узлов решений, узлов событий и конечных узлов (исходов).

Построение дерева решений

Свойства деревьев решений:

- имеются вершины альтернативных решений, которые обычно отображаются в виде прямоугольников;
- имеются вершины событий, которые обычно отображаются с помощью окружностей;
- имеются вершины конечных решений (листья), которые обычно отображаются с помощью прямоугольников, имеющих другой цвет, чем вершины альтернативных решений;
- вершина альтернативных решений вместе с дугами, выходящими из данной вершины, называется вилкой решений;
- вершина событий вместе с дугами, выходящими из данной вершины, называется стохастической вилкой (стохастической - имеющей вероятностный характер, так как в процессе принятия решения учитываются вероятности каждого из событий);
- каждой дуге вилки решений приписаны весы, которые характеризуют приобретения или убытки при реализации данного решения;
- каждой дуге стохастической вилки приписываются весы, которые характеризуют вероятность соответствующего события, а также приобретения и убытки при реализации данного события.

Анализ дерева решений

При решении прикладных задач методом дерева решений поступают следующим образом:

- на первом шаге проходят от начальной вершины к такой конечной вершине (листу), которой соответствует наибольшее приобретение, не учитывая при этом вероятности событий, предшествующих решениям;
- на втором шаге учитывают вероятности событий, предшествующих решениям; приобретения на вершинах, отображающих конечные решения, рассчитывают с учётом вероятностей событий, предшествующих решениям.

Задача

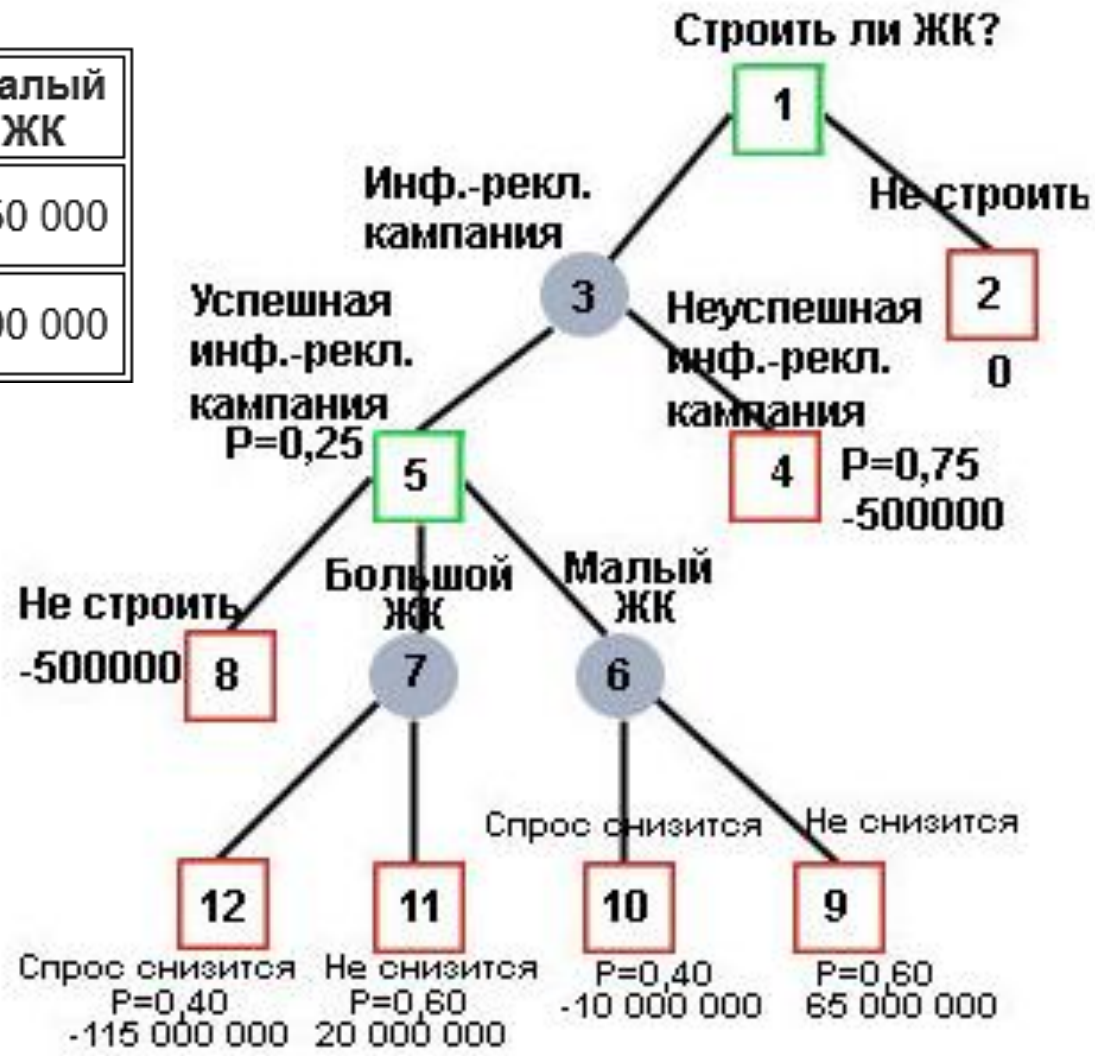
Решить **методом дерева решений** задачу прогнозирования приобретений и убытков при альтернативах решений.

- Строительная фирма собирается принять решение о строительстве жилого комплекса (ЖК) в элитном районе. Сначала требуется принять решение: проводить ли информационно-рекламную кампанию. Она стоит 500000 условных единиц (у.е.). Опыт показывает, что лишь в 25 % случаев этот шаг обеспечивает успех на рынке.
- Если информационно-рекламная кампания успешна, требуется принять решение: строить ли большой или малый ЖК. Строительство малого ЖК обойдётся в 50000000, при этом можно построить 300 квартир. Строительство большого ЖК обойдётся в 200000000, при этом можно построить 900 квартир.
- Имеются исследования прогноза спроса. Они показывают, что существует вероятность в 40 % того, что произойдёт падение спроса на элитное жильё.
- По предварительным расчётам, средние цены на квартиры будут определяться следующим образом:

- Рассчитано, что расходы фирмы перед и в период продажи всех квартир в ЖК составят 5000000, независимо от величины ЖК.
- Требуется принять решение: проводить ли информационно-рекламную кампанию и начинать строительство ЖК.

Строим дерево решений

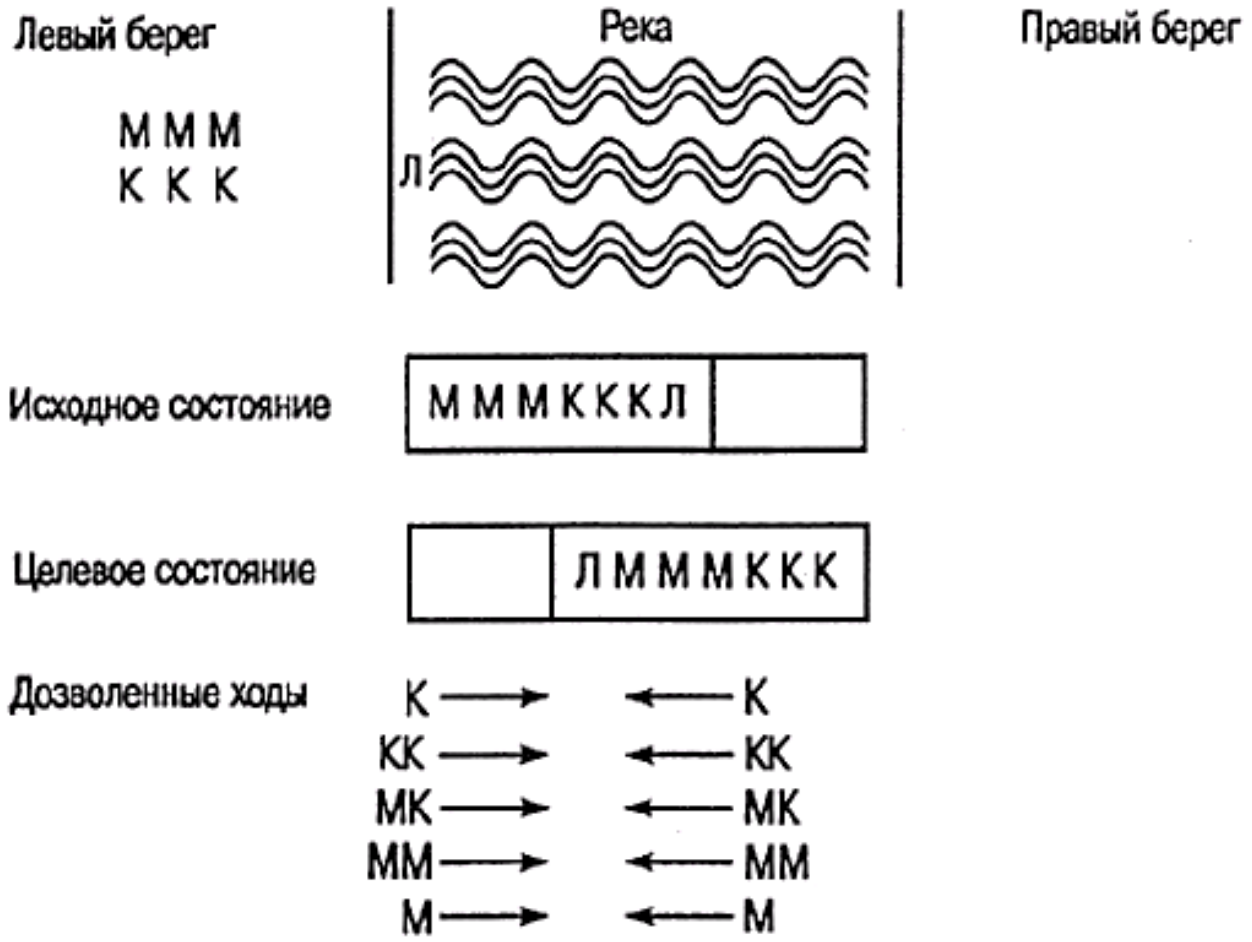
	Большой ЖК	Малый ЖК
Спрос снизится	100 000	150 000
Спрос не снизится	250 000	400 000



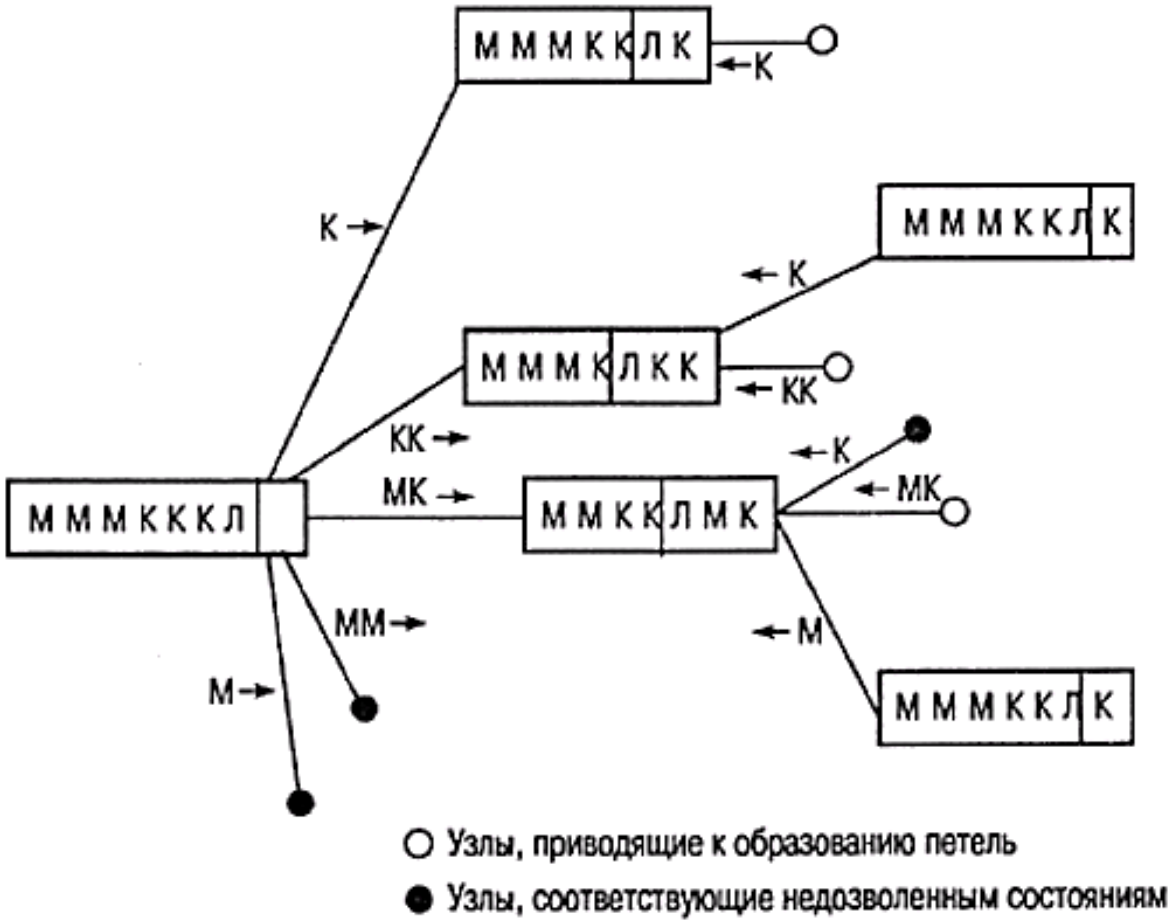
Дерево игры

- **Теория игр** занимается попыткой выразить особенности столкновения интересов через действия, которые имеют место в салонных играх. Правила любой салонной игры устанавливают последовательность вполне определённых ходов, причём каждый ход для данного игрока соответствует выбору одной из множества альтернатив.
- В игре нужно учитывать все ходы, предшествующие данному ходу, и все возможные ходы, следующие за ним. Таким образом, значение каждого хода в игре зависит от других ходов. Если создать отвлечённое представление всех ходов игры и указать, какие выборы привели к каждому ходу, то можно узнать отвлечённую связь каждого хода со всеми другими ходами, которые повлияли на него или на которые он может повлиять.

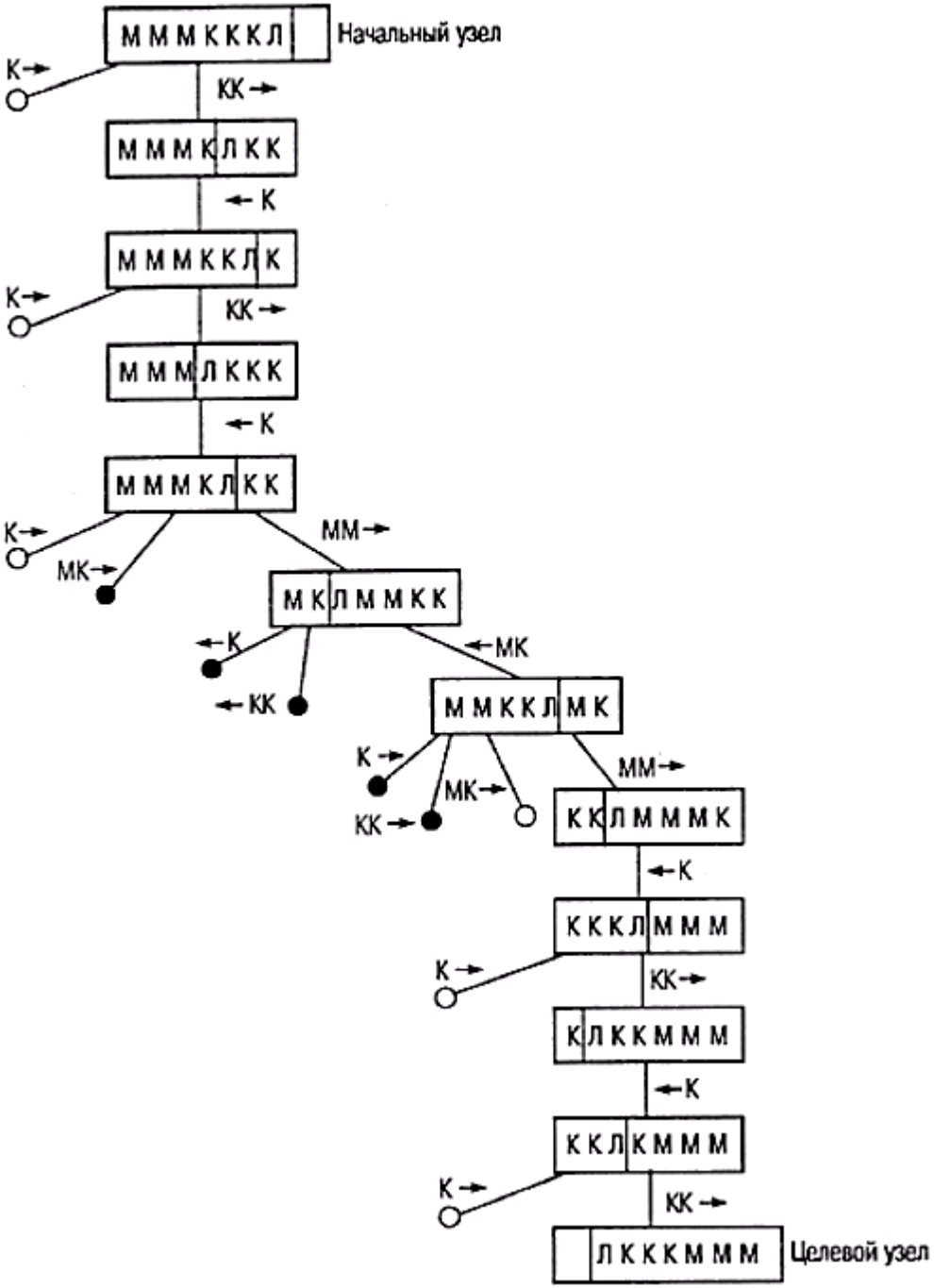
Головоломка "миссионеры и каннибалы"



Построение пространства поиска в головоломке "миссионеры и каннибалы"



Законченное пространство поиска в головоломке "миссионеры и каннибалы", сформированное алгоритмом поиска в глубину.



Головоломка "Восьмерка"

2	8	3
1	6	4
7		5

Исходное состояние (Пример)

Целевое состояние

1	2	3
8		4
7	6	5

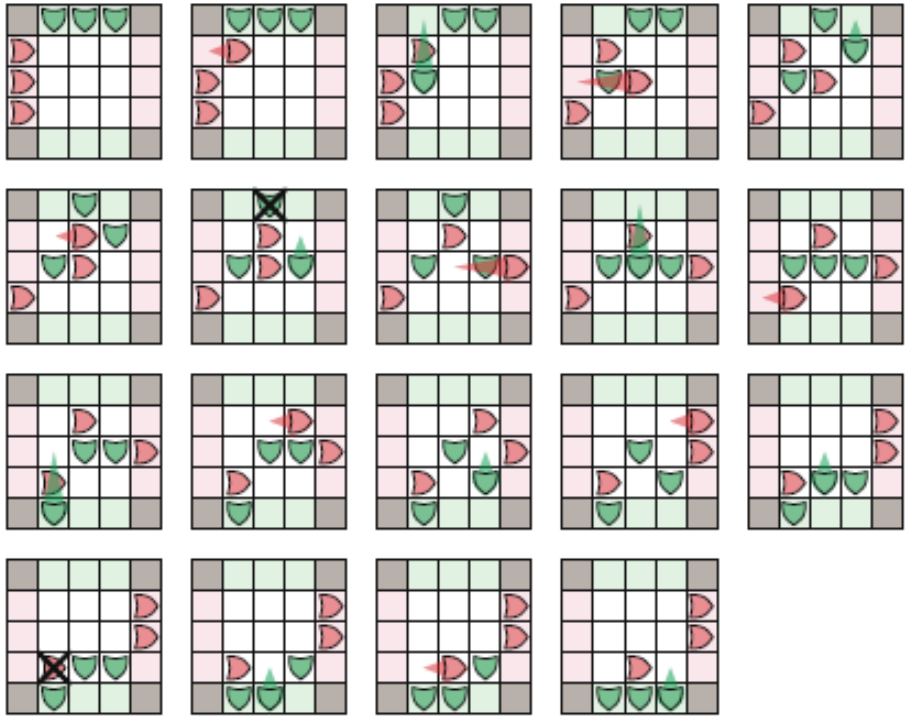
Дозволенные ходы: освободить поле выше
ниже
слева
справа

Ограничения: нельзя освободить поле по диагонали

В отличие от задачи о миссионерах и каннибалах, эту головоломку можно решить за приемлемое время методом "слепого" поиска. Дело в том, что головоломка имеет только 9! состояний и, следовательно, можно использовать для поиска очередного хода оценочную функцию по методике "восхождения на гору".

Игра <без имени>

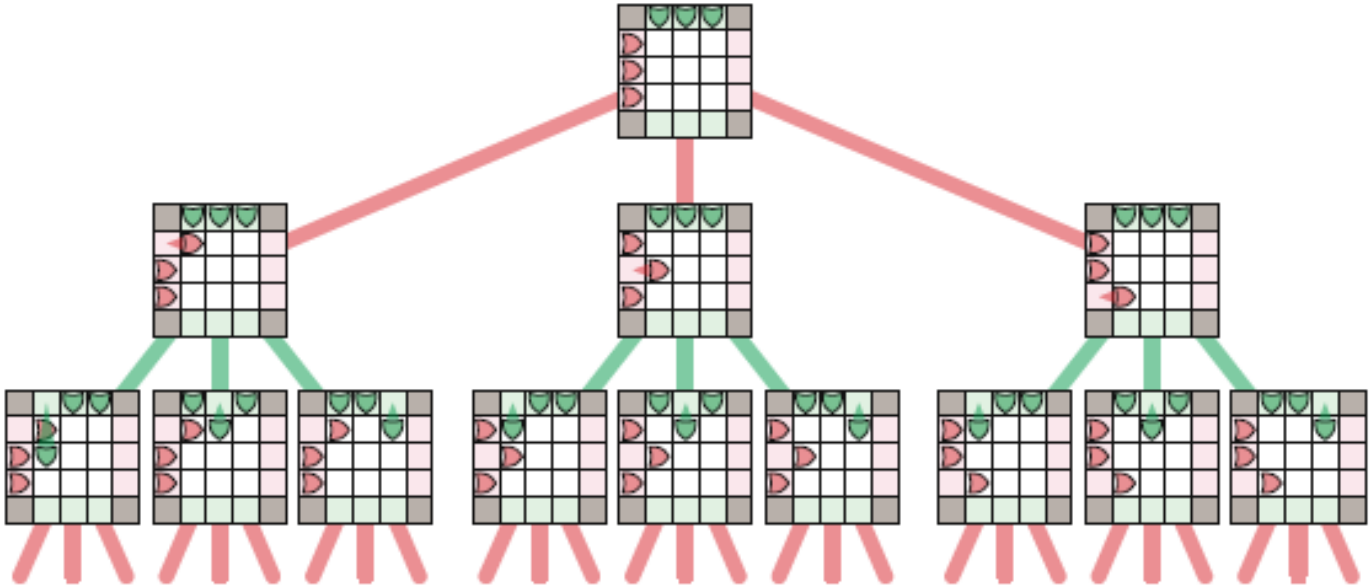
- Два игрока (ходят по очереди), размер поля n*n.
- Первый игрок начинает у левого края доски и перемещается по горизонтали вправо. Может передвигать одну фишку на соседнее свободное поле справа или перепрыгнуть через одну фишку второго игрока на свободное поле справа.
- Второй игрок начинает у верхнего края доски и перемещается вниз (аналогично).
- Если нет разрешенных ходов и прыжков, игрок пропускает ход.
- Задача: первым переместить все свои фишки к противоположному краю доски.



Дерево игры

```
PlayAnyGame(X, player):
  if player has already won in state X
      return Good
  if player has already lost in state X
      return Bad
  for all legal moves X ⇝ Y
      if PlayAnyGame(Y, ¬player) = Bad
          return Good        ⟪X ⇝ Y is a good move⟫
  return Bad                 ⟪There are no good moves⟫
```

Алгоритм поиска с возвратом



- Стратегия:
  - Состояние хорошее (good), если текущий игрок уже выиграл или если текущий игрок своим ходом может перевести игру в плохое состояние,
  - Состояние игры плохое (bad), если текущий игрок уже проиграл или если любой его ход ведет к хорошему состоянию для другого игрока.

Упорядочивание дуг и вершин орграфа

Под упорядочиванием вершин связного орграфа без контуров, то есть циклических цепей, понимают такое разбиение его вершин на группы, при котором:

- 1) вершины первой группы не имеют предшествующих вершин, а вершины последней группы последующих;
- 2) вершины любой другой группы не имеют предшествующих в следующей группе;
- 3) вершины одной и той же группы дугами не соединяются.

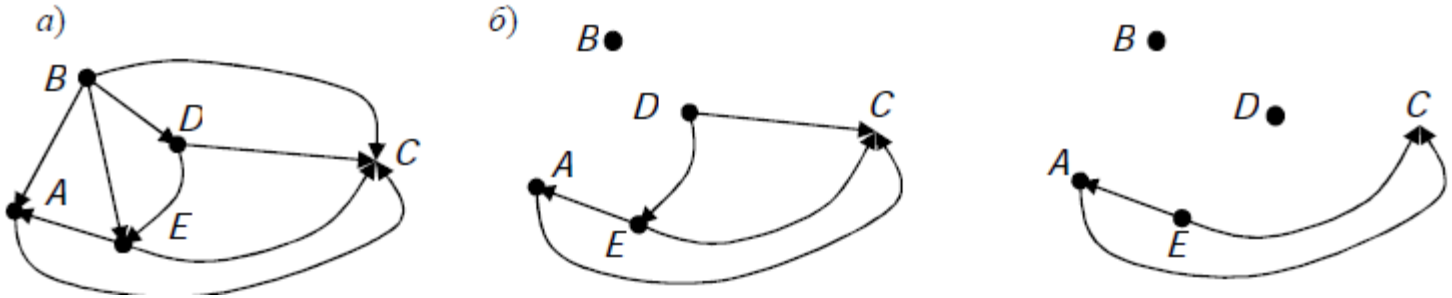
Такое разбиение всегда возможно. В результате подобного упорядочивания получается граф, изоморфный исходному. Упорядочивание элементов выполняется графическим или матричным способом.

Алгоритм Фалкерсона

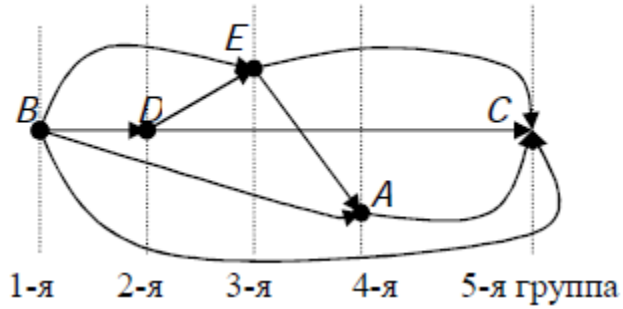
Графический способ носит название алгоритма Фалкерсона и состоит из следующих шагов.

- 1. Найти вершины графа, в которые не входит ни одна дуга. Они образуют первую группу. Пронумеровать вершины группы в произвольном порядке.
- 2. Вычеркнуть все пронумерованные вершины и дуги, из них исходящие. В получившемся графе найдется, по крайней мере, одна вершина, в которую не входит ни одна дуга. Этой вершине, входящей во вторую группу, присвоить очередной номер и так далее. Второй шаг повторять до тех пор, пока не будут упорядочены все вершины.

Аналогичным образом упорядочиваются дуги орграфа.

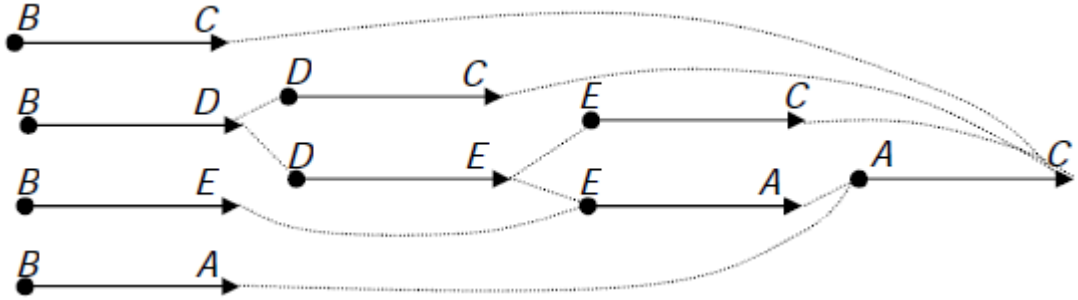


Алгоритм Фалкерсона



$$P = \begin{array}{c} \\ A \\ B \\ C \\ D \\ E \end{array} \begin{array}{c} \begin{array}{ccccc} A & B & C & D & E \end{array} \\ \begin{pmatrix} 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 \end{pmatrix} \end{array}$$

	A	B	C	D	E
v_1	2	0	4	1	2
v_2	1	-	3	0	1
v_3	1	-	2	-	0
v_4	0	-	1	-	-
v_5	-	-	0	-	-
Группа	4	1	5	2	3



Определение экстремальных путей на графах

- Задачи о кратчайших путях

Пусть задана *сеть* (ориентированный граф), в котором выделены две вершины – исток и сток. Для каждой дуги заданы числа, называемые длинами дуг.

Длиной пути (контура) называется сумма длин входящих в него дуг (если длины дуг не заданы, то длина пути (контура) определяется как число входящих в него дуг). Задача заключается в поиске кратчайшего пути (пути минимальной длины) от входа до выхода сети .

- Задачи об экстремальных контурах

Задачи об экстремальных контурах

Задача поиска контура минимальной длины.
Если известно, что искомый контур содержит некоторую вершину, то нужно определить кратчайшей путь от этой вершины до нее же, применяя описанные выше алгоритмы. Так как в общем случае контур минимальной длины может проходить через любую вершину графа, то находятся контуры минимальной длины, проходящие через каждую вершину, и среди них выбирается кратчайший.

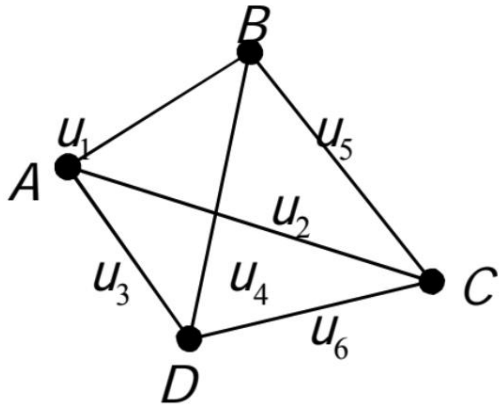
Задача поиска контура минимальной средней длины
поиск контура, для которого минимально отношение его длины к числу содержащихся в нем дуг.

Задача о ранце
Пусть имеется *n* предметов, которые могут быть полезны в походе. Полезность n-го предмета оценивается числом a, вес предмета (или его объем) – m. Суммарный вес, который может нести турист (объем рюкзака), ограничен величиной *R* . Требуется найти набор предметов, обладающий максимальной суммарной полезностью и удовлетворяющий ограничению.

Путь максимальной эффективности
Пусть задана сеть, в которой для каждой дуги *(i ; j)* определены два числа интерпретируемые как эффект при осуществлении соответствующей операции и затраты на эту операцию. Эффективность пути определяется как отношение его эффекта к затратам Задача заключается в поиске пути максимальной эффективности.

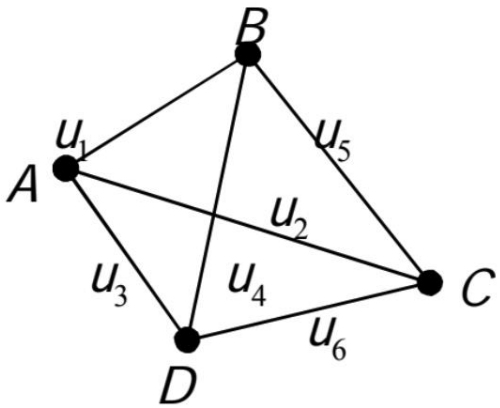
Выявление маршрутов с заданным количеством ребер

- *Теорема*. Для определения количества маршрутов, состоящих из ***к*** ребер (дуг), необходимо возвести в ***к-ую*** степень матрицу смежности вершин. Тогда соответствующий элемент даст количество маршрутов длины ***к*** (состоящих из ***к*** ребер).



$$\begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix} = \begin{pmatrix} 3 & 2 & 2 & 2 \\ 2 & 3 & 2 & 2 \\ 2 & 2 & 3 & 2 \\ 2 & 2 & 2 & 3 \end{pmatrix}.$$

Выявление маршрутов с заданным количеством ребер



$$\begin{pmatrix} 0 & u_1 & u_2 & u_3 \\ u_1 & 0 & u_5 & u_4 \\ u_2 & u_5 & 0 & u_6 \\ u_3 & u_4 & u_6 & 0 \end{pmatrix} \cdot \begin{pmatrix} 0 & u_1 & u_2 & u_3 \\ u_1 & 0 & u_5 & u_4 \\ u_2 & u_5 & 0 & u_6 \\ u_3 & u_4 & u_6 & 0 \end{pmatrix} =$$

$$= \begin{pmatrix} u_1u_1 + u_2u_2 + u_3u_3 & u_2u_5 + u_3u_4 & u_1u_5 + u_3u_6 & u_1u_4 + u_2u_6 \\ u_5u_2 + u_4u_3 & u_1u_1 + u_5u_5 + u_4u_4 & u_1u_2 + u_4u_6 & u_1u_3 + u_5u_6 \\ u_5u_1 + u_6u_3 & u_1u_2 + u_6u_4 & u_2u_2 + u_5u_5 + u_6u_6 & u_2u_3 + u_5u_4 \\ u_4u_1 + u_2u_6 & u_3u_1 + u_5u_6 & u_3u_2 + u_4u_5 & u_3u_3 + u_4u_4 + u_6u_6 \end{pmatrix}.$$

Выявление маршрутов с заданным количеством ребер

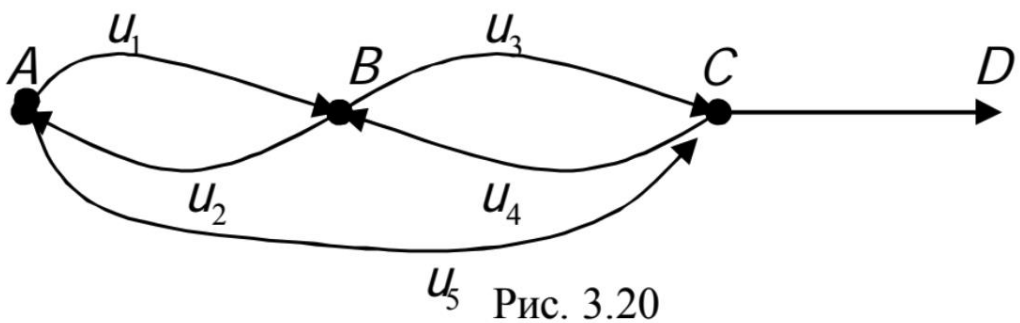


Рис. 3.20

$$P^2 = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 2 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \quad P^3 = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 2 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \cdot \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix} =$$

$$= \begin{pmatrix} 1 & 2 & 2 & 1 \\ 2 & 1 & 2 & 1 \\ 0 & 2 & 1 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}.$$

Выявление маршрутов с заданным количеством ребер

Следствие 1. **В графе G мощности n тогда и только тогда существует (x_i, x_j)-маршрут $(x_i \neq x_j)$, когда (i, j)-й элемент матрицы $P + P^2 + \ldots + P^{n-1}$ не равен нулю.**

Следствие 2. **В графе G мощности n тогда и только тогда существует цикл, содержащий вершину x_i, когда (i, i)-й элемент матрицы $P + P^2 + \ldots + P^n$ не равен нулю.**

$$P^4 = \begin{pmatrix} 2 & 3 & 3 & 2 \\ 1 & 4 & 3 & 2 \\ 2 & 1 & 2 & 1 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

$$A = P + P^2 + P^3 = \begin{pmatrix} 2 & 4 & 4 & 2 \\ 3 & 3 & 4 & 2 \\ 1 & 3 & 2 & 2 \\ 0 & 0 & 0 & 0 \end{pmatrix}, \quad B = P + P^2 + P^3 + P^4 = \begin{pmatrix} 4 & 7 & 7 & 4 \\ 4 & 7 & 7 & 4 \\ 3 & 4 & 4 & 3 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

Метод Шимбелла

1. Операция умножения двух величин a и b при возведении матрицы в степень соответствует их алгебраической сумме, то есть

$$\begin{cases} a \cdot b = b \cdot a \Rightarrow a + b = b + a, \\ a \cdot 0 = 0 \cdot a = 0 \Rightarrow a + 0 = 0. \end{cases} \qquad (3.8.1)$$

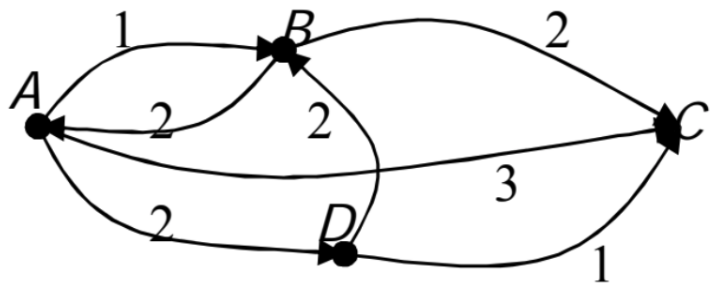
2. Операция сложения двух величин a и b заменяется выбором из этих величин минимального (максимального) элемента, то есть

$$a + b = b + a = \min(\max)\{a, b\}, \qquad (3.8.2)$$

нули при этом игнорируются. Минимальный или максимальный элемент выбирается из ненулевых элементов. Нуль в результате операции (3.8.2) может быть получен лишь тогда, когда все элементы из выбираемых – нулевые.

$$\omega_{ij}^{(2)} = \min(\max)\left\{\left(\omega_{i1}^{(1)} + \omega_{1j}^{(1)}\right), \left(\omega_{i2}^{(1)} + \omega_{2j}^{(1)}\right), \ldots, \left(\omega_{in}^{(1)} + \omega_{nj}^{(1)}\right)\right\}$$

Пример



$$P^2 = \begin{pmatrix} 0 & 1 & 3 & 2 \\ 2 & 0 & 2 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 2 & 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} 0 & 1 & 3 & 2 \\ 2 & 0 & 2 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 2 & 1 & 0 \end{pmatrix} = \begin{pmatrix} 3 & 4 & 3 & 0 \\ 0 & 2 & 5 & 4 \\ 0 & 0 & 0 & 0 \\ 4 & 0 & 4 & 0 \end{pmatrix}.$$

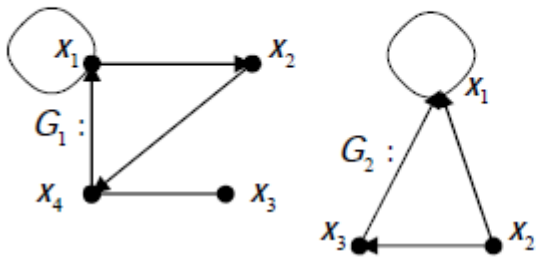
$$P^3 = \begin{pmatrix} 3 & 4 & 2 & 0 \\ 0 & 2 & 5 & 4 \\ 0 & 0 & 0 & 0 \\ 4 & 0 & 4 & 0 \end{pmatrix} \cdot \begin{pmatrix} 0 & 1 & 3 & 2 \\ 2 & 0 & 2 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 2 & 1 & 0 \end{pmatrix} = \begin{pmatrix} 6 & 4 & 6 & 5 \\ 4 & 6 & 4 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 5 & 7 & 6 \end{pmatrix}$$

СПАСИБО
ЗА ВНИМАНИЕ

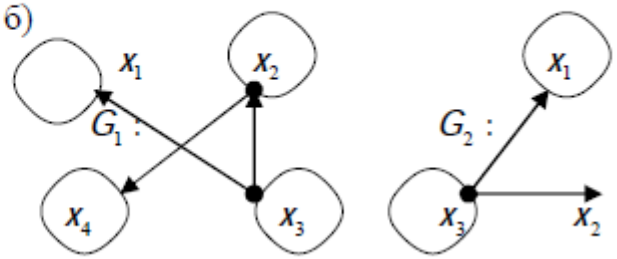
$$\text{а)}\ \begin{pmatrix} 1 & 1 & 2 & 0 & 0 \\ 1 & 0 & 1 & 3 & 0 \\ 2 & 1 & 1 & 0 & 2 \\ 0 & 3 & 0 & 1 & 1 \\ 0 & 0 & 2 & 1 & 0 \end{pmatrix};$$

3.9.4. На рис. 3.23 приведены графы G_1 и G_2. Найти $G_1 \cup G_2$ и $G_1 \times G_2$ если:

а)

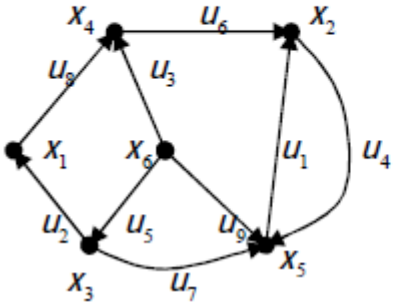


б)



3.9.2. Для изображенных графов на рис. 3.22 составить матрицы смежности вершин, смежности дуг и инциденций.

а)



б)

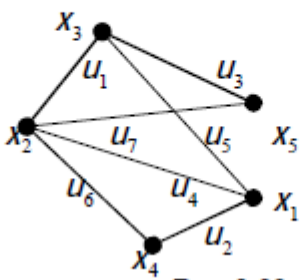
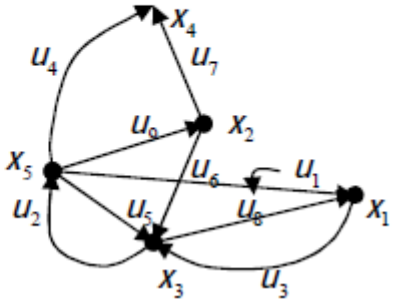


Рис. 3.22

в)